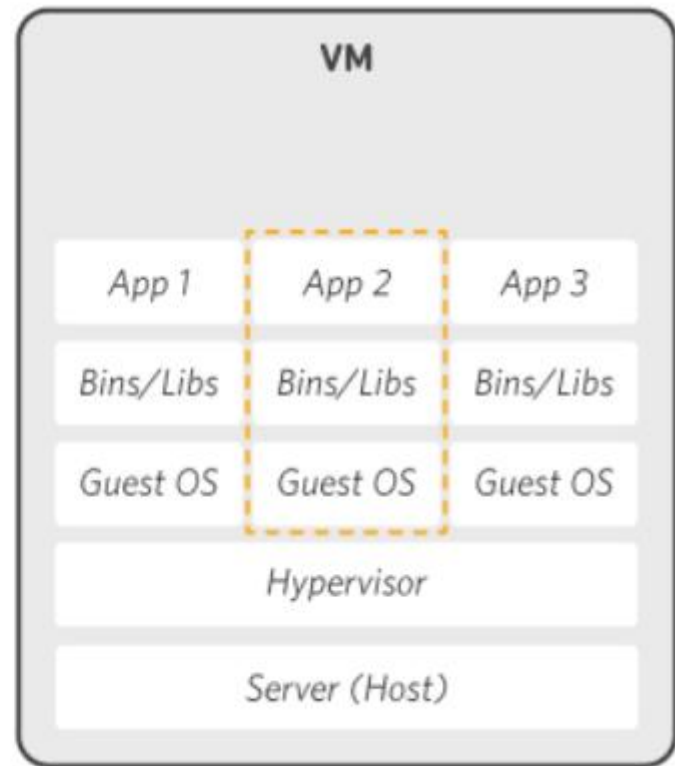
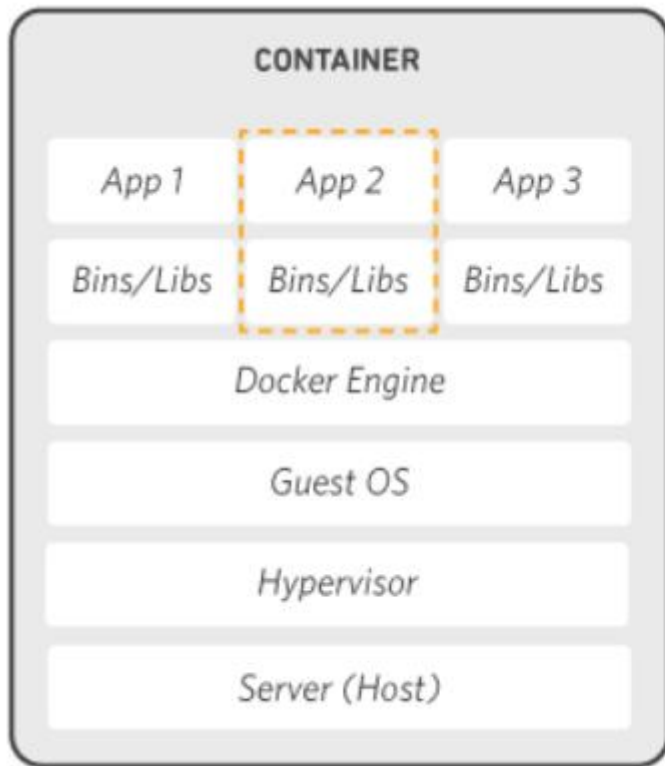


도커 (Docker)

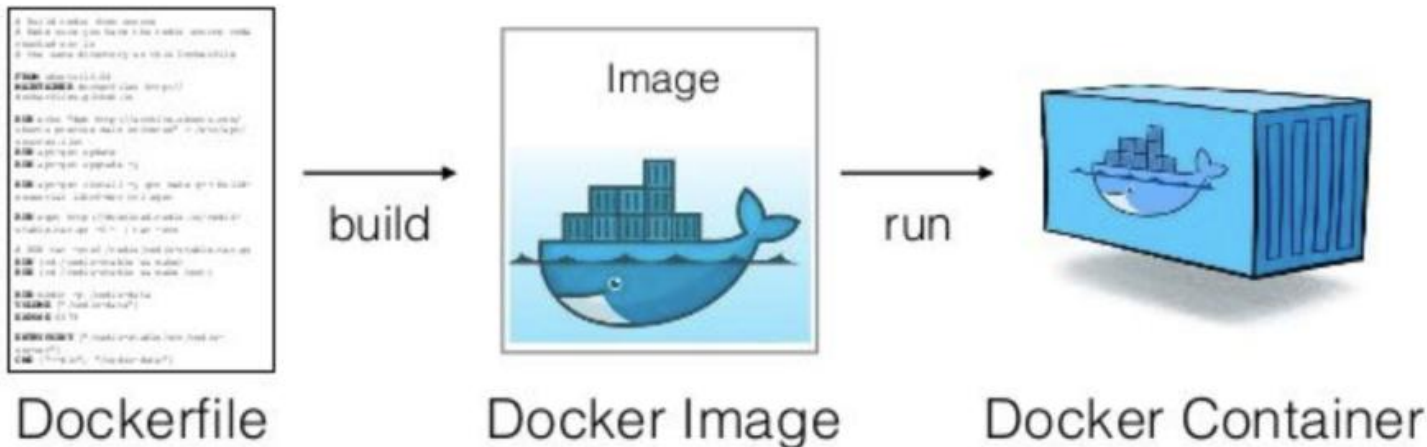
- 애플리케이션을 신속하게 구축, 테스트 및 배포할 수 있는 소프트웨어 플랫폼
- Docker는 소프트웨어를 컨테이너라는 표준화된 유닛으로 패키징
- 컨테이너에는 라이브러리, 시스템 도구, 코드, 런타임 등 소프트웨어를 실행하는 데 필요한 모든 것이 포함
 - ▣ 프로그램의 배포 관리 단순화
- Docker를 사용하면 컨테이너를 매우 가벼운 모듈식 가상 머신처럼 다룰 수 있다.
- 컨테이너를 구축, 배포, 복사하고 한 환경에서 다른 환경으로 이동

Docker은 컨테이너를 위한 운영 체제!

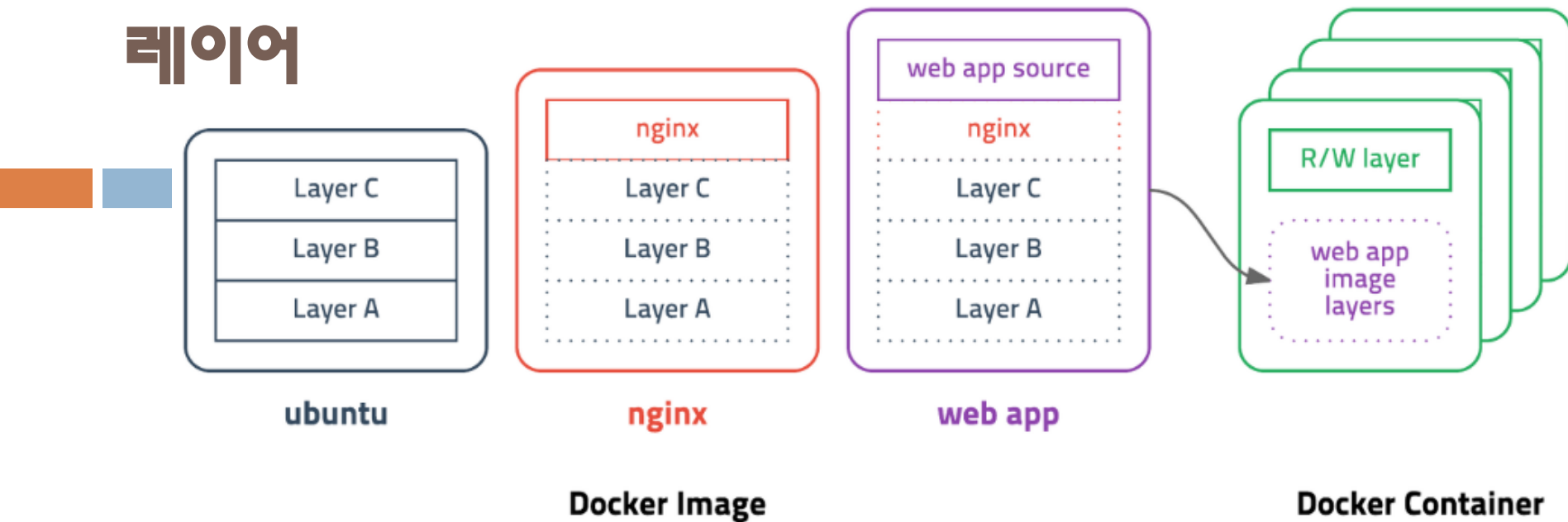


Docker image

- 도커에서 서비스 운영에 필요한 서버 프로그램, 소스코드 및 라이브러리, 컴파일된 실행 파일을 묶는 형태
- 특정 프로세스를 실행하기 위한(즉, 컨테이너 생성(실행)에 필요한) 모든 파일과 설정값(환경)을 지닌 것
 - Ubuntu이미지는 Ubuntu를 실행하기 위한 모든 파일을 가지고 있음
 - Oracle이미지는 Oracle을 실행하는데 필요한 파일과 실행명령어, port정보 등을 모두 가지고 있음



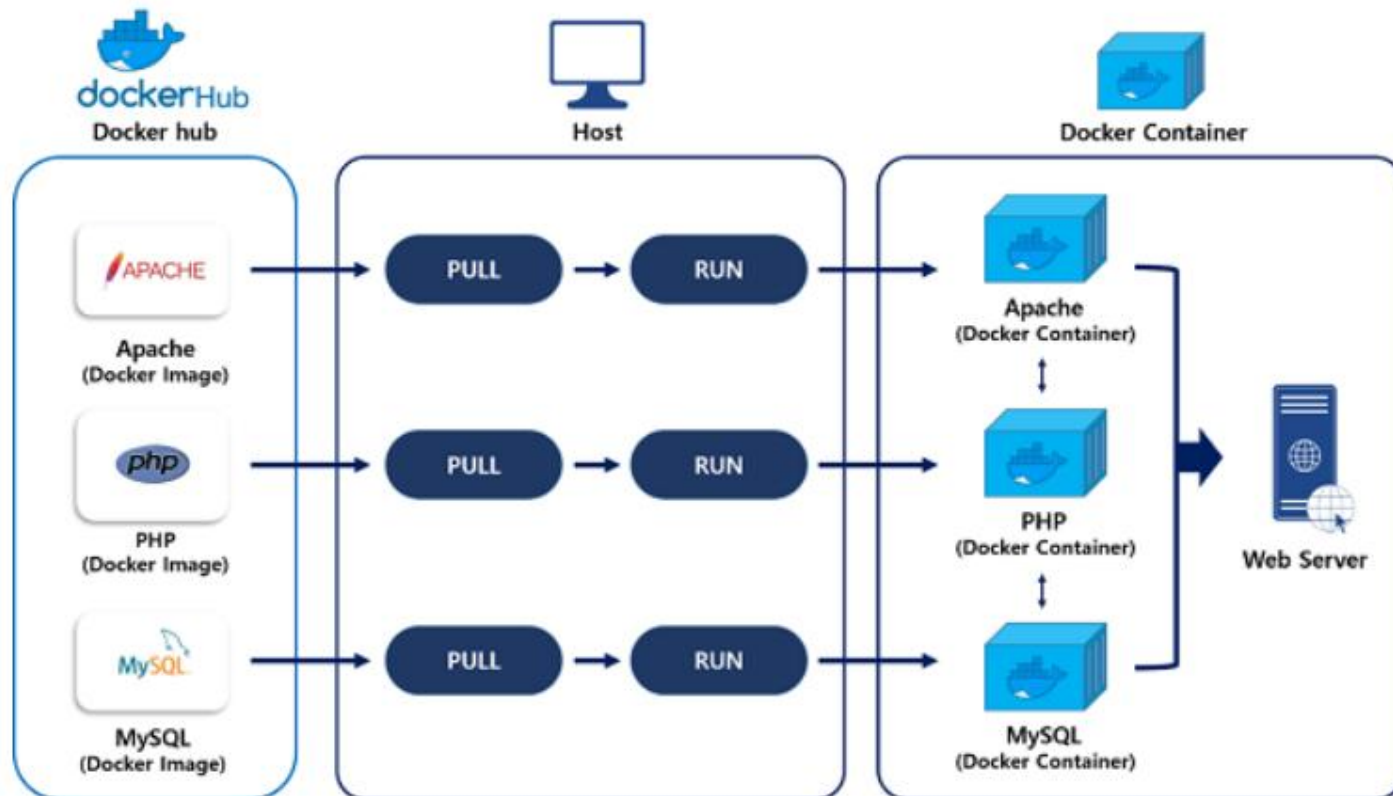
레이어



- 기존 이미지에 파일 하나 추가했다고 수백메가를 다시 다운로드받으면 매우 비효율적
- 레이어란 기존 이미지에 추가적인 파일이 필요할 때 다시 다운로드받는 방법이 아닌 해당 파일을 추가하기 위한 개념
- 이미지는 여러 개의 읽기 전용 레이어로 구성되고 파일이 추가되거나 수정되면 새로운 레이어가 생성

컨테이너

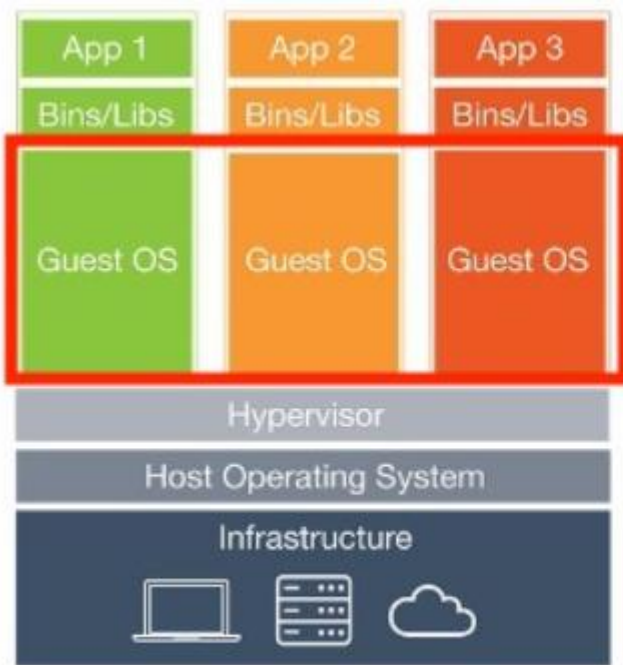
- 이미지(Image)를 실행한 상태로, 응용프로그램의 종속성과 함께 응용프로그램 자체를 패키징 or 캡슐화하여 격리된 공간에서 프로세스를 동작시키는 기술



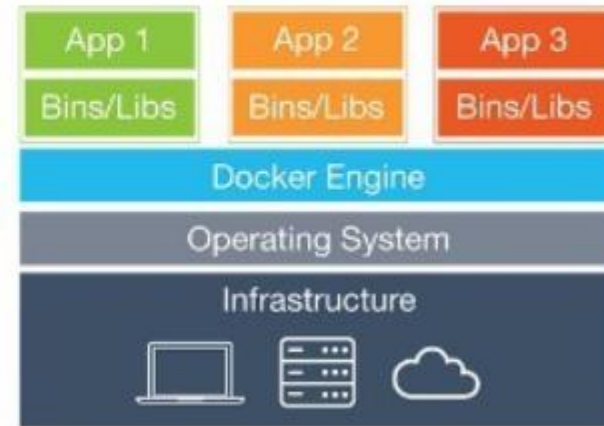
도커의 특징

- 가상머신처럼 독립적으로 실행되지만, 가상머신보다 빠르고 쉽고 효율적이다.
 - ▣ 도커는 컴퓨터 자원을 그대로 사용한다. (가상머신은 독립 된 메모리를 할당)
- 확장성과 이식성
 - ▣ 도커가 설치되어 있다면 어디서든 컨테이너를 실행할 수 있다.
 - ▣ 오픈 소스이기에 특정 회사나 서비스에 종속적이지 않다.
- 표준성
 - ▣ 도커를 사용하지 않는 경우, 각각의 언어로 만든 서비스들의 배포 방식은 모두 다르다.
 - ▣ 도커는 컨테이너라는 표준으로 서버를 배포하므로 모든 서비스들의 배포 과정이 동일해진다.
- 이미지
 - ▣ **Dockerfile**을 이용하여 이미지를 만들고 처음부터 재현 가능하다.
- 자원관리
 - ▣ 컨테이너는 삭제 후 새로 만들면 모든 데이터가 초기화된다. (제거가 쉽다.)

도커와 VM의 차이



Virtual Machines



Containers

- 게스트OS의 유무
- 자원의 효율성 측면에서 차이가 난다.
- VM에는 위에서 언급한 Guest OS가 깔린다.
 - ▣ VM는 하나씩 늘 때마다 OS를 위한 자원을 할당
- 도커의 컨테이너에는 Guest OS를 설치하지 않는다.
 - ▣ 도커는 어플리케이션을 구동하는데 필요한 모든 패키지만 있으면 컨테이너를 구동

Docker 자주 사용하는 명령어

□ Docker System 정보

- ▣ docker version : Docker 버전 및 상세정보 확인
- ▣ docker -v : Docker 버전 및 빌드 버전을 한줄로 간단하게 확인.
- ▣ docker system info : Docker의 시스템 전체 정보 확인.
- ▣ docker system event : 서버로 부터 실시간 이벤트를 받는다.
- ▣ docker system df : 도커 디스크의 사용량 확인.
- ▣ docker login -u [ID] : 도커 허브에 로그인

Docker Image 명령어

- docker의 image 목록을 확인
 - ▣ docker images
 - ▣ docker image ls
- 모든 이미지 표시
 - ▣ docker images -a
- image 상세 정보 표시
 - ▣ docker image inspect [이미지명]
- docker hub image 검색
 - ▣ docker search [옵션] <검색어>
 - ▣ docker search mysql

□ 옵션 사용 예제

- ▣ 자동빌드 설정, star수 15개 이상인 mysql 이미지
- ▣ `docker search --filter is-automated=true --filter starts=15 mysql`

□ 옵션

- ▣ `automated=[true/false]` : 자동화 빌드 설정 여부
- ▣ `no-trunc=false`: 모든 결과물 출력 (No Filtering)
- ▣ `filter start=[start 수] <검색어>` : star 수 만큼 검색.
- ▣ `filter is-official=true` : 공식 이미지만 검색
- ▣ `limit=[갯수]` : 갯수만큼의 검색 결과물을 검색 (default = 25개)

Docker Container

- 현재 실행중인 컨테이너 확인
 - ▣ `docker ps`
- 실행중, 중지중 컨테이너 리스트 확인
 - ▣ `docker ps -a`
- 컨테이너 단건 삭제
 - ▣ `docker rm [컨테이너 ID]`
- 컨테이너 다건 삭제
 - ▣ `docker rm [컨테이너 ID], [컨테이너 ID]`
- 컨테이너 전체 삭제
 - ▣ `docker rm $(docker ps -a -q)`

컨테이너 접속

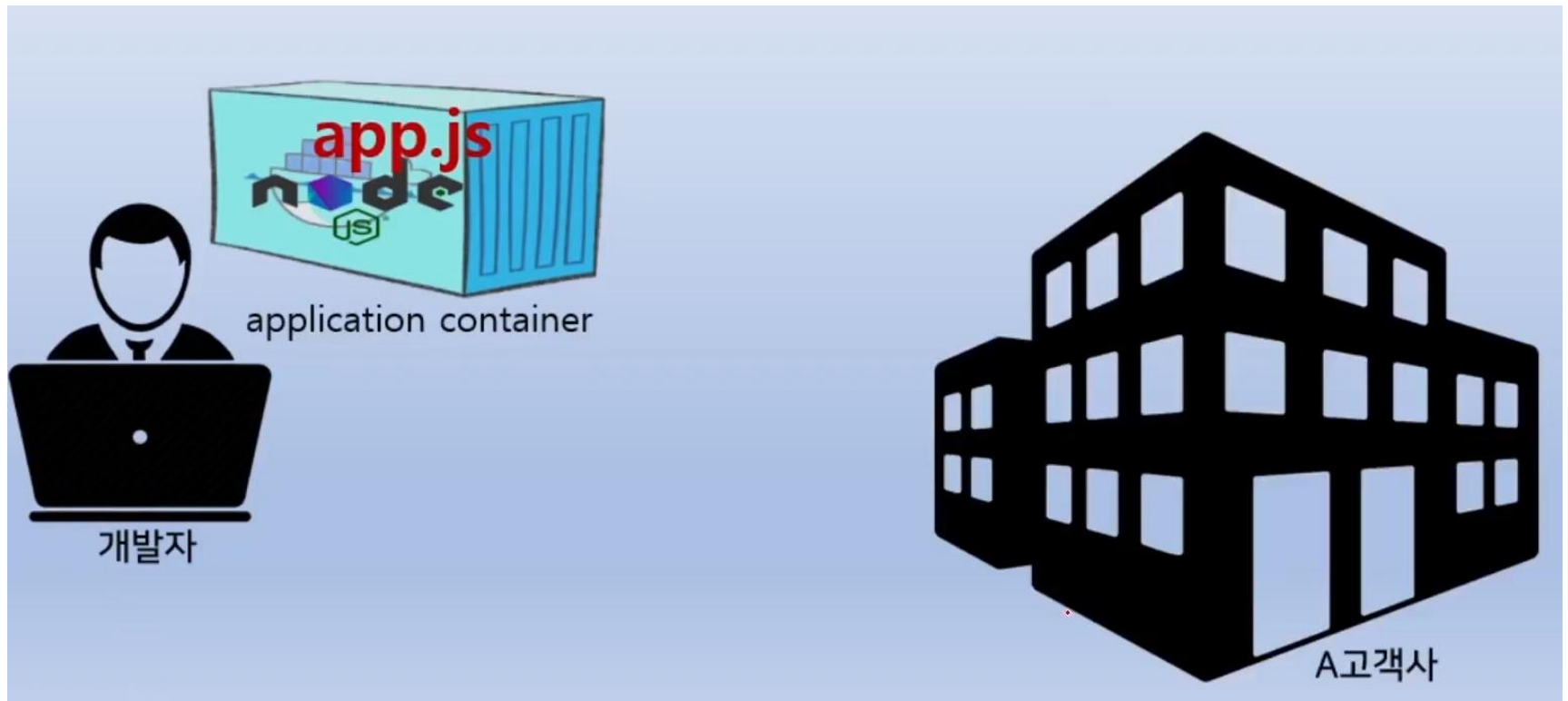
- 실행되고 있는 컨테이너에 접속 - **attach**
- 컨테이너 **run**시 **/bin/bash** 사용하지 않았다면 접속 불가
 - ▣ **docker attach** [컨테이너명, ID]
- 실행중인 컨테이너에 접속, 명령수행가능(일시적) - **exec**
- **exit**로 종료
 - ▣ **docker exec -it** [컨테이너명, ID] **/bin/bash**
- 컨테이너 시작
 - ▣ **docker start** [컨테이너명, ID]
- 컨테이너 중지
 - ▣ **docker stop** [컨테이너명, ID]
- 컨테이너 재시작
 - ▣ **docker restart** [컨테이너명, ID]

Docker Run - 컨테이너화

- local있는 image를 사용할것인가, docker hub에 존재하는 image 를 사용할것인가
- image -> Container 생성 및 실행
 - ▣ docker run [options] image[:tag, @digest] [command] [args]
- Local
 - ▣ docker run -d mysql:latest
- Docker Hub
- docker run [options] [docker hub ID]/[이미지명]
 - ▣ docker run -d test123/mysql

- 가장 많이 쓰이는 옵션 및 실전 예제
- docker hub id가 test123에서 mysql 이미지를 컨테이너화
- 컨테이너를 터미널에 연결허용(-it), 백그라운드에서 동작(-d), 포트포워딩(-p)
- 컨테이너의 이름은 testMysql로 지정.
 - ▣ docker run -it -d -p 8080:8080 --name testMysql test123/mysql
- 옵션
 - ▣ -it : 표준 입력(STDIN) 허용 및 터미널 연결 가능설정 (-i, -t 옵션을 같이 사용)
 - ▣ -d : 컨테이너를 백그라운드에서 실행 (Ex) SpringBoot 서버 , DB{MySQL, Oracle})
 - ▣ -p : 컨테이너의 포트를 포워딩 하는 옵션 [host port:컨테이너 port]
 - 8080:3000 - 8080포트로 접속하면 컨테이너의 3000 포트로 포워딩.
 - ▣ --name : 생성될 컨테이너의 이름을 지정.
- 컨테이너 이름 변경
 - ▣ docker rename [현재 컨테이너명] <변경할 컨테이너 이름>

- 고객사마다 환경을 다르게 설정해서 똑같이 동작시킴

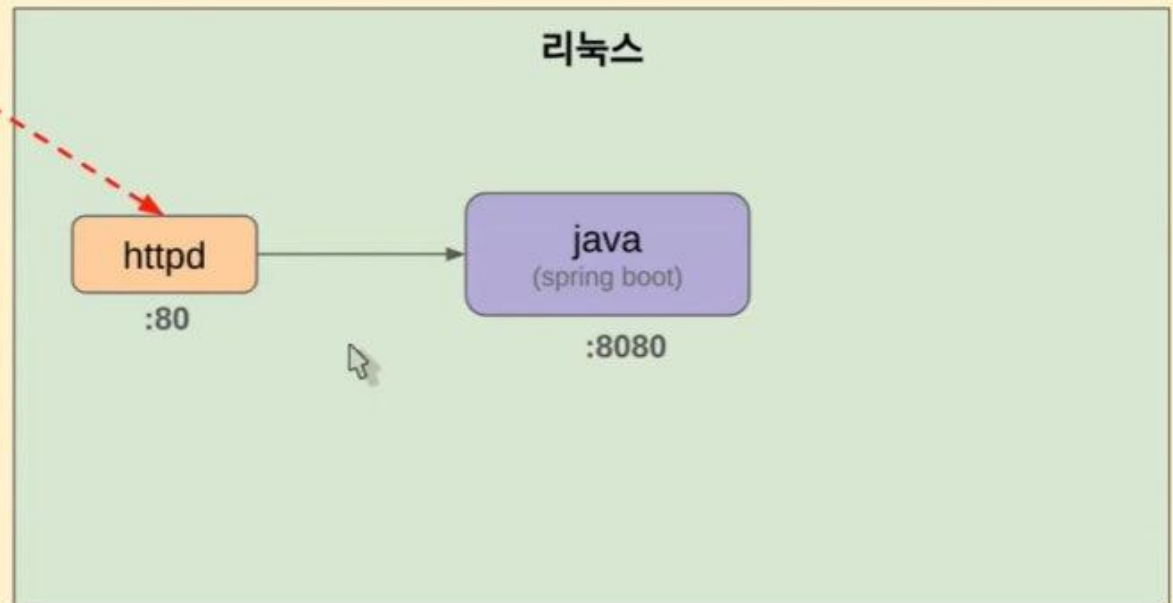


포트 포워딩

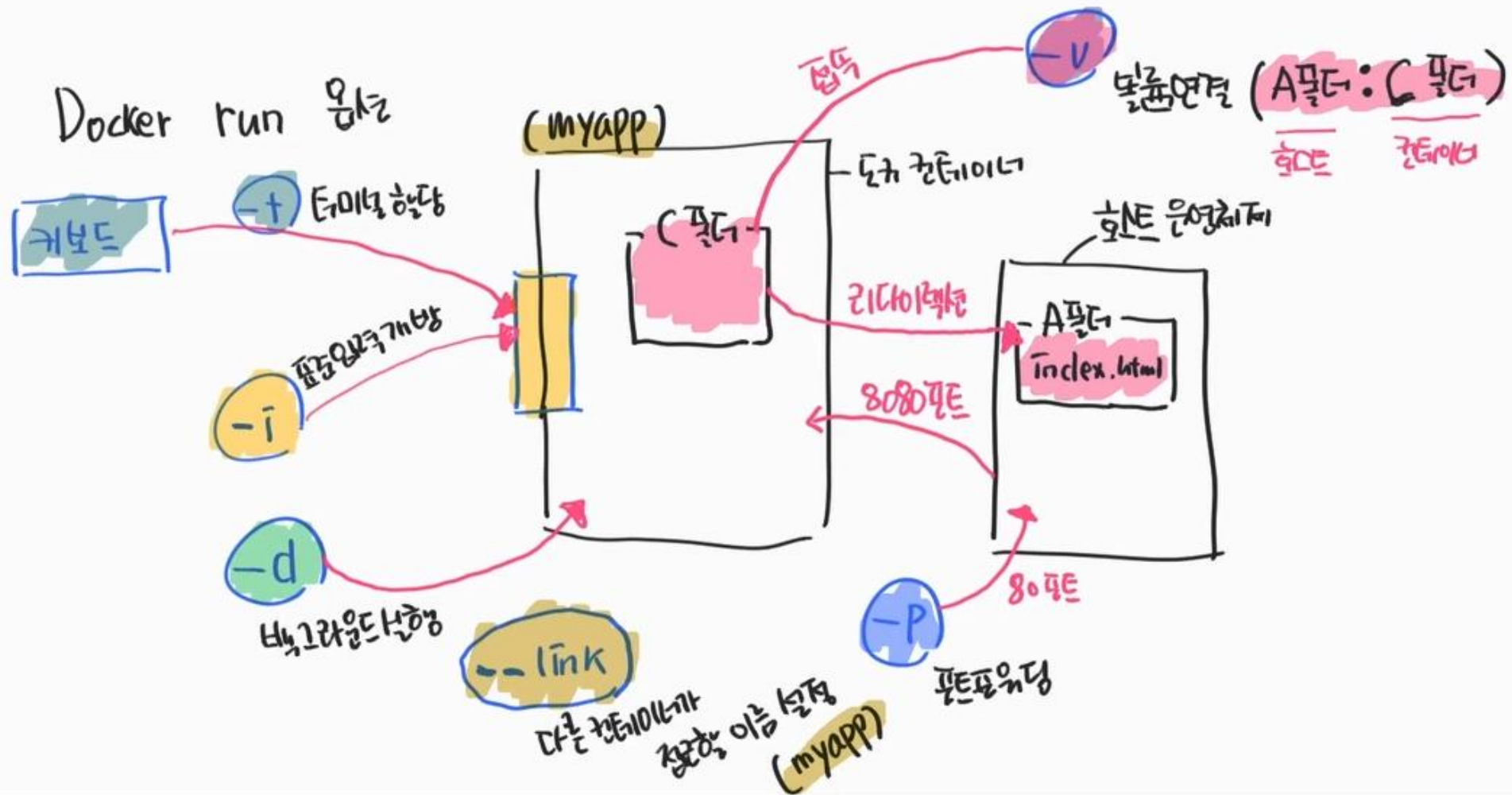


Apache(httpd) 포트 포워딩 구조

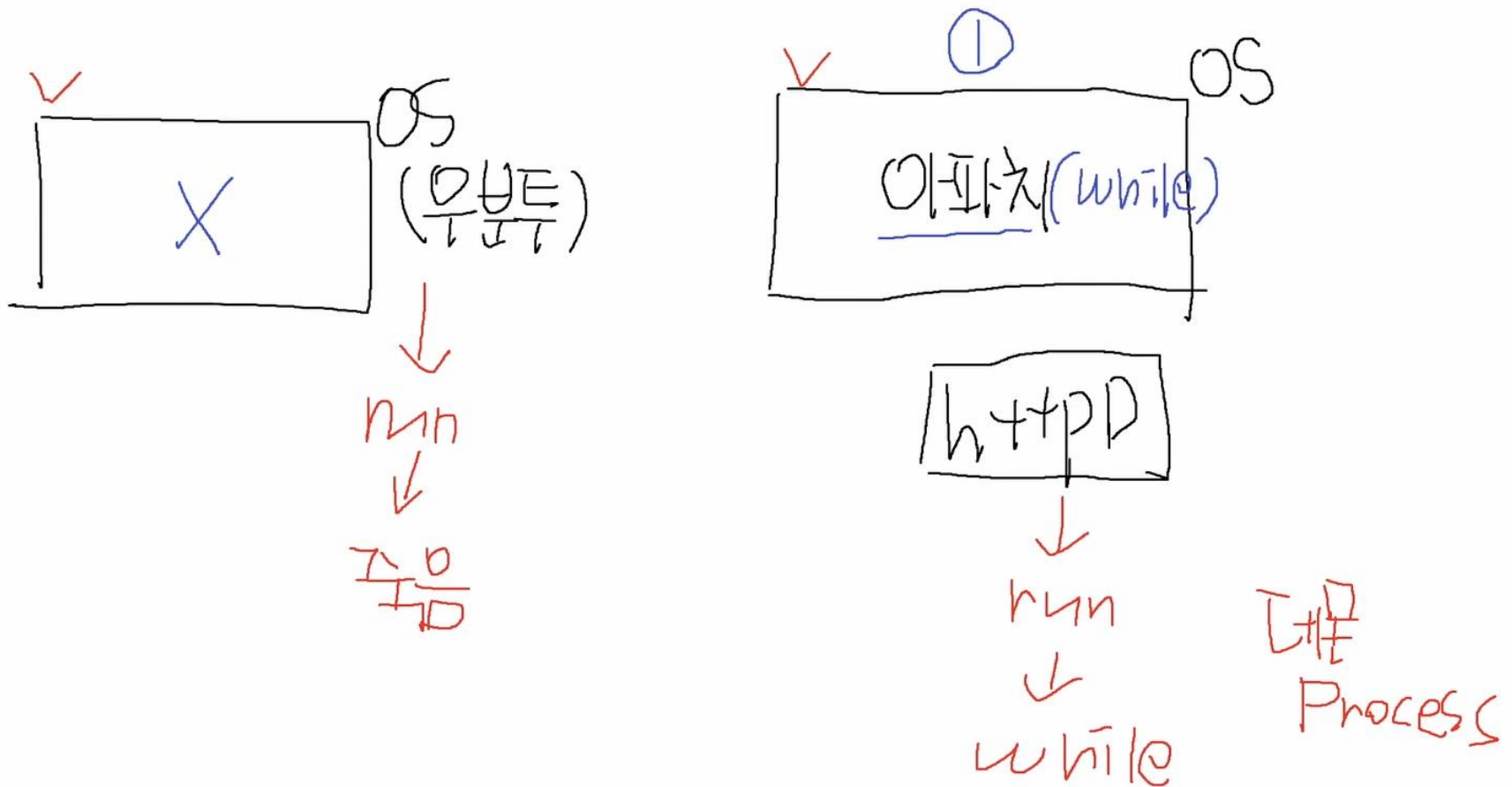
:80 접속



docker run 옵션



- ❑ `docker run -d --name myubuntu ubuntu`
`docker run -dit --name myubuntu ubuntu`



1. os (ubuntu)

```
docker run -dit ubuntu bash
```

```
docker attach 컨테이너ID
```

2. while process (httpd)

```
docker run -d -p 8080:80 httpd
```

```
docker exec -it 컨테이너ID bash
```

```
docker run -dit -p 8080:80 httpd
```

Dockerfile 로 image 생성(빌드) / 삭제

- 기본 커맨드
 - ▣ `docker build -t [dockerHub ID]/[이미지명]:[태그명] [DockerFile위치]`
 - ▣ `-t` 옵션 : 만들어질 이미지를 의미
- 태그 지정X 시 :latest 지정됨
 - ▣ `docker build -t [dockerHub ID]/[이미지명] .`
- 이미지만 삭제
 - ▣ `docker rmi [이미지 ID]`
- 컨테이너 + 이미지 강제 삭제
 - ▣ `docker rmi -f [이미지 ID]`

ENTRYPOINT와 CMD 함께 사용하기

```
FROM centos:7
ENTRYPOINT ["echo", "Hello, "]
CMD ["Darwin"]
```

```
docker build -t hello:together .
```

```
docker run hello:together
```

Hello, Darwin # 매개변수 없이 컨테이너를 실행하면 CMD에 지정한 인자 사용

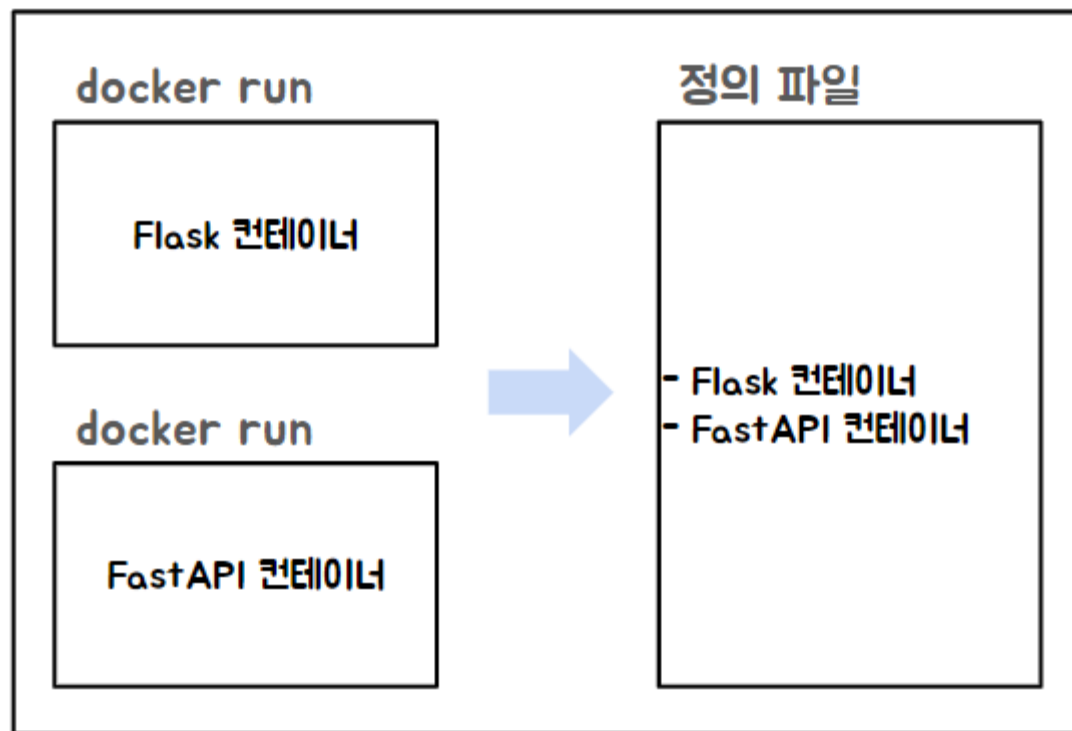
```
docker run hello:together world
```

Hello, world # 매개변수 넘겨주면 CMD 기본값은 오버라이딩되어 무시됨

- 항상 실행해야 하는 명령을 사용하여 실행 가능한 도커 이미지를 빌드할 때는 ENTRYPOINT를 사용
- CMD 명령어는 도커 컨테이너가 실행될 때 command line에 명시적으로 인자값을 지정하지 않는 경우에 기본 명령어 역할을 하는 인자를 설정하는 데에 사용

도커 컴포즈

- 시스템 구축과 관련된 명령어를 하나의 텍스트 파일(정의 파일)에 기재해 명령어 한 번에 시스템 전체를 실행하고 종료와 폐기까지 한 번에 하도록 도와주는 도구



도커 컴포즈 커맨드

- up 커맨드 (생성, 시작)
 - ▣ docker run 커맨드와 비슷
 - ▣ 정의 파일에 기재된 내용대로 이미지를 내려받고 컨테이너를 생성 및 실행
 - ▣ 정의 파일에는 네트워크나 볼륨에 대한 정의도 기재할 수 있어서 주변 환경을 한꺼번에 생성
- down 커맨드 (정지, 삭제)
 - ▣ 컨테이너와 네트워크를 정지 및 삭제
 - ▣ 볼륨과 이미지는 삭제 X
 - ▣ 컨테이너와 네트워크 삭제 없이 종료만하고 싶다면 stop 커맨드를 사용